

Vstup a výstup

Karel Richta a kol.

Katedra technických studií
Vysoká škola polytechnická v Jihlavě

© Karel Richta, 2022

Základy strukturovaného programování, ZSP
10/2022, Lekce 4

<https://moodle.vspj.cz/course/view.php?id=202262>



Vstup a výstup

- Je realizován pomocí standardních knihoven:
 - <stdio.h>** pro C a
 - <cstdio>** nebo **<iostream>** pro C++.
- Může být neformátovaný (binární) nebo formátovaný (textový).
- **Neformátovaný** vstup/výstup - reprezentace dat v paměti a na vstupu/výstupu je stejná. Čtecí/zápisové operace nezasahují do zpracovávané posloupnosti bytů (slabik).
- **Formátovaný** vstup/výstup - paměťová (binární) reprezentace je odlišná od textové reprezentace na vstupu/výstupu. Výstupní operace provádí konverzi z paměťové (binární) reprezentace na text. Čtecí operace převádějí textovou reprezentaci na paměťovou, vnitřní binární reprezentaci.

Příklad (celá čísla)

- Příklad neformátovaného (binárního) vstupu/výstupu:

Hodnota	v paměti (pro Sparc)	v souboru
1	00 00 00 01 (4B)	00 00 00 01 (4B)
1000	00 00 03 E8 (4B)	00 00 03 E8 (4B)
1000000	00 0F 42 40 (4B)	00 0F 42 40 (4B)

- Příklad formátovaného vstupu/výstupu:

Hodnota	v paměti (pro Sparc)	v souboru (např. pro ASCII)
1	00 00 00 01	31 (1B)
1000	00 00 03 E8	31 30 30 30 (4B)
1000000	00 0F 42 40	46 34 32 34 30 (5B)

Vstup a výstup

<stdio.h>

- Standardní knihovna <stdio.h> (v C++ se používá varianta cstdio)
- Vstup a výstup:
 - textový (formátovaný)
 - binární (neformátovaný)
- Textový vstup a výstup
 - standardní: printf, putchar, scanf, getchar
 - z/do souboru: fprintf, putc, fscanf, getc
 - z/do paměti: sprintf, sscanf
 - *přímo na obrazovku (není dle normy)*
- Standardní textové soubory:
 - stdin vstupní, klávesnice, lze přesměrovat
 - stdout výstupní, obrazovka, lze přesměrovat
 - stderr výstupní, obrazovka

Standardní výstup

`<stdio.h>`

```
int putchar(int c);
```

- výstup znaku c;
- při chybě vrací EOF (-1), jinak c

```
int puts(char *s);
```

- výstup řetězce s + nový řádek

```
int printf(char *format, ...);
```

- výstup hodnot různých typů;
- vrací počet vytisknutých znaků
- formát:
 - řetězec - popisuje typ a tvar argumentů
 - výstupní konverze: začíná znakem %
 - ostatní znaky kopírovány na výstup
- konverze:
 - % [*příznaky*] [*šířka*] [. *přesnost*] [*velikost*] *typ*

Výstupní konverze

`<stdio.h>`

<code>%d</code> or <code>%i</code>	dekadické číslo se znaménkem	392
<code>%u</code>	dekadické číslo bez znaménka	7235
<code>%o</code>	oktalové číslo bez znaménka	610
<code>%x</code> , <code>%X</code>	hexadecimální číslo bez znaménka	7fa, 7FA
<code>%f</code> , <code>%F</code>	dekadické desetinné (lower, upper)	392.65, 392.65
<code>%e</code>	vědecká notace (mantisa/exponent), malá	3.9265e+2
<code>%E</code>	vědecká notace (mantisa/exponent), velká	3.9265E+2
<code>%g</code>	použij kratší z reprezentací: <code>%e</code> nebo <code>%f</code>	392.65
<code>%G</code>	použij kratší z reprezentací: <code>%E</code> nebo <code>%F</code>	392.65
<code>%a</code>	hexadecimální desetinné číslo, malá	-0xc.90fep-2
<code>%A</code>	hexadecimální desetinné číslo, velká	-0XC.90FEP-2

Výstupní konverze (pokr.)

%c	Znak (character)	a
%s	Řetězec (string)	sample
%p	Adresa (pointer)	b8000000
%n	Nic se netiskne, odpovídající argument musí být adresa celočíselné buňky, do které se zapíše počet dosud vytištěných znaků	
%%	výstup znaku %	%

Parametry konverzí

%[příznaky][šířka][.přesnost][velikost]konverze

- **Příznaky:** - zarovnání doleva, + výstup znaménka vždy, 0 – výplň znaky 0,
- **Šířka:** minimální šířka výstupu (doplněno mezerami, nezkracuje se) nebo * - pak je šířka dána dalším argumentem
- **Přesnost:** pro celá čísla minimální počet znaků, pro desetinná počet míst za tečkou
- **Velikost:** h – short, l – long, ll – long long, L – long double

Příklady výstupní konverze

`<stdio.h>`

Příklady (místo mezery je použito podtržítko)

konverze	hodnota	výstup
%d	-300	-300
%-5i	-5	-5__
%5x	0xABC	__abc
%#09X	0x12AB	0X00012AB
%6.2f	4.352	__4.35
%+6.0f	100.44	__+100
%10.3E	100.44	_1.004E+02
%*.*f	7, 2, 100.123	_100.12
%c	'a'	a
%4s	"hello"	hello
%.4s	"hello"	hell
%-10.4s	"hello"	hell_____

Standardní vstup

`<stdio.h>`

`int getchar(void) ;`

- vstup znaku (při chybě, konci souboru EOF)

`int gets(char *s) ;`

- vstup řetězce do konce řádku (nebezpečí: není omezena délka řetězce)

`int scanf(char *format, ... ) ;`

- vstup hodnot různých typů; argumenty musejí být ukazatelé
- vrací počet úspěšně zkonvertovaných a přiřazených parametrů (s výjimkou konverze n)
- formát
 - platí obdobná pravidla jako u printf
 - konverze uvedeny znakem %
 - mezery ignorovány (s výjimkou konverze c, [a n)
 - ostatní znaky srovnávány se vstupem
- konverze:
- % [*] [šířka] [velikost] *typ*

Vstupní konverze

`<stdio.h>`

* položka je konvertována, ale není přiřazena

Šířka číslo maximální celková šířka (bez úvodních mezer)

Velikost viz velikost u printf

l u konverze f, e, E, g, G: parametr typu double *(jinak float *)

Typ

d	dekadické celé číslo; parametr typu int *
i	celé číslo v libovolné soustavě (8, 10, 16); parametr typu int *
o	oktalové celé číslo; parametr typu unsigned int *
x, X	šestnáctkové celé číslo; parametr typu unsigned int *
f, e, E, g, G	racionální číslo; parametr typu float * !!!
s	posloupnost nemezerových znaků; parametr typu char *; řetězec musí být dostatečně dlouhý!
c	posloupnost znaků (implicitní šířka je 1); parametr typu char *
[	přeskočí posloupnost vyjmenovaných znaků

Příklad

```
#include <stdio.h>

int main(void){
    int x; float f; char name[50];

    scanf("%2d%f %[ 0123456789]s", &x, &f, name);
}
```

pro vstup:

56789 0123 56a72

nastaví:

x == 56, f == 789.0, name == 0123 56\0

Vstup a výstup v C

Nevýhody funkcí **printf** a **scanf**:

- formátovací řetězec musí odpovídat parametrům,
- překladač nemá obecně možnost kontrolovat, zda skutečně odpovídá (omezeně se o to snaží např. gcc),
- chyba v párování způsobí chybu konverze či dokonce pád programu.

Výhody funkcí **printf** a **scanf**:

- trochu kompaktnější zápis,
- trochu snazší lokalizace (lze lépe oddělit řetězce od programu).
- Neformátovaný výstup (binární) se provádí pomocí jiných funkcí - práce se soubory v C – viz popis funkcí **fopen**, **fclose**, **fread**, **fwrite**, ...

Formátovaný výstup v C

Formátovaný výstup v C - `<stdio.h>` – funkce `printf(formát,...)`:

- formát - formátovací řetězec:
- text zobrazený beze změny,
- substituce (uvozené znakem %, také tzv. konverze).
- parametry pro substituce,
- substituce ve formátovacím řetězci musí typově odpovídat parametrům,
- návratová hodnota – počet úspěšně vypsáných znaků.

Příklad:

```
int a = 10;
double b = 2.5;
const char * c = "Hello";
printf ("a= %d, b= %f, c= %s\n", a, b, c);
```

Formátovaný vstup v C

Formátovaný vstup v C - <stdio.h> – funkce scanf(formát,...):

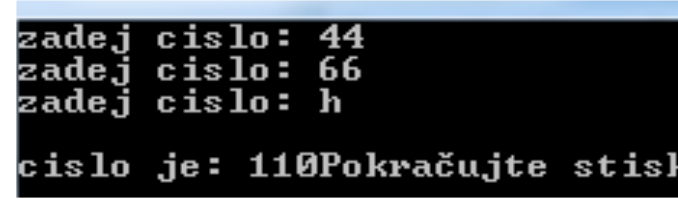
- formát - formátovací řetězec,
- substitute (uvozené znakem %, také tzv. konverze),
- adresy proměnných pro uložení hodnot,
- substitute ve formátovacím řetězci musí typově odpovídat parametrům,
- návratová hodnota – počet úspěšně použitých konverzí.

Příklad:

```
int a;  
double b;  
char c[40];  
scanf ("%d %lf %s\n", &a, &b, c);  
scanf ("%d %lf %39s\n", &a, &b, c); // pole je adresa
```

Ošetření vstupních hodnot

```
int main()
{
    int cislo;
    int sum = 0;
    while (1) {
        printf("zadej cislo: ");
        if (scanf("%d", &cislo) != 1)
            break;
        sum+=cislo;
    }
    printf("\ncislo je: %d", sum);
    system("pause");
    return 0;
}
```



```
zadej cislo: 44
zadej cislo: 66
zadej cislo: h
cislo je: 110Pokračujte stisk
```


Formátovaný vstup a výstup v paměti

`<stdio.h>`

```
int sscanf(char *Buffer, char *Format, ...);
```

- vstup z řetězce Buffer

```
int sprintf(char *Buffer, char *Format, ...);
```

- výstup do řetězce Buffer

Příklady:

```
sscanf(Retezec, "%s = %f", &S, &F);
```

```
Retezec == "Výsledek = 4.52"
```

```
S == "Výsledek"
```

```
F == 4.52
```

```
sprintf(Jmeno, "Temp%d", I);
```

```
I == 3
```

```
Jmeno == "Temp3"
```

Soubory

- Soubory představují informaci uloženou na vnějším médiu.
- Obsah souboru můžeme interpretovat binárně nebo textově.
- Při binárním zpracování interpretujeme obsah souboru jako posloupnost bytů, tj. hodnot v rozsahu 0..255 (`unsigned char`).
- Při textovém zpracování interpretujeme obsah souboru jako posloupnost znaků (`char`). V textových souborech interpretujeme jistou kombinaci bytů jako znak konce řádku. Tato kombinace závisí na systému ovládání souborů v operačním systému:
 - LF (`\n` = 0x0A) UNIX,
 - CR LF (`\r\n` = 0x0D 0x0A) Windows, DOS,
 - CR (`\r` = 0x0D) Mac.
- V paměti je konec řádku vždy LF (`'\n'`) :
 - C/C++ knihovna provádí automatickou konverzi na LF (při čtení nahradí posloupnost CR LF (CR) znakem LF, při zápisu nahradí znak LF znaky CR LF (CR) dle systému.

Soubory tvořící standardní prostředí

- Jsou to „soubory“ **stdin** (vstup), **stdout** (výstup) a **stderr** (chybový výstup).
- Jsou připojeny k programu automaticky, není potřeba se o nic starat.
- Pracuje se s nimi v textovém režimu.
- Příklad: kopírování ze standardního vstupu na standardní výstup:

```
#include <stdio.h>

int main() {
    int znak;
    while ((znak = getchar()) != EOF) {
        putchar(znak);
    }
    return 0;
}
```

- Otázka: Proč je proměnná znak typu int?

Soubory mimo standardní prostředí

- Mimo standardních souborů lze k programu připojit libovolné množství dalších souborů – jejich zpracování ale musíme explicitně předepsat.
- Informace o souborech se udržují v paměti ve strukturách nazývaných řídicí blok souboru a označených identifikátorem FILE.
- Pro standardní soubory se tyto struktury vytvoří automaticky, pro ostatní soubory je musí vytvořit programátor a svázat s nějakým souborem pomocí funkce `fopen(jméno, mód)`. Naopak uvolnit je lze pomocí funkce `fclose(soubor)`.
- Módem se myslí způsob zpracování souboru: textový režim (implicitně), nebo binární, dále soubor určený ke čtení, zápisu, nebo pro obojí.
- Pro formátovaný vstup/výstup (textové soubory), či pro binární soubory jsou v knihovně `stdio` funkce, které popíšeme později.
- Pro textový vstup z/výstup do souboru můžeme použít alternativu `fprintf` a `fscanf`.
- Příklad: kopírování textového souboru.

Otevření a zavření souboru

```
FILE *fopen(char *Jmeno, char *Mod) ;
```

```
<stdio.h>
```

- otevření souboru specifikovaného Jména
- Jmeno - může obsahovat cestu k souboru (pozor: \\)
- Mod - způsob otevření souboru; řetězec složený ze znaku:
 - r otevření na začátku (čtení)
 - w existující obsah vymazán (psaní)
 - a otevření na konci (zápis na konec)
 - b binární soubor (jinak textový)
 - + update
- povolené kombinace:
 - "r", "w", "a" textový soubor
 - "rb", "wb", "ab" binární soubor
 - "r+", "w+", "a+" textový soubor, update
 - "rb+" = "r+b", "wb+" = "w+b", "ab+" = "a+b" binární soubor, update

```
int fclose(FILE *Soubor) ;
```

- zavření souboru

Soubory mimo standardní prostředí

- Příklad: kopírování textového souboru (ReadMe.txt) do souboru (kopie.txt):

```
#include <stdio.h>

int main() {
    FILE *vstup, *vystup;
    int znak;
    vstup = fopen("ReadMe.txt", "r");
    vystup = fopen("kopie.txt", "w");
    while ((znak = fgetc(vstup)) != EOF) {
        fputc(znak, vystup);
    }
    fclose(vstup);
    fclose(vystup);
    return 0;
}
```

Kopírování souborů – po znacích

```
#include <stdio.h>

int main(int argc, char *argv[]) {
    FILE *vstup, *vystup;
    int znak;
    if (argc != 3) {
        fprintf(stderr, "Použití: %s vstup výstup\n", argv[0]);
        return 1;
    }
    vstup = fopen(argv[1], "r");
    if (vstup == NULL) {
        fprintf(stderr, "Vstupní soubor %s nelze otevřít\n", argv[1]);
        return 2;
    }
    vystup = fopen(argv[2], "w");
    if (vystup == NULL) {
        fprintf(stderr, "Výstupní soubor %s nelze otevřít\n", argv[2]);
        return 3;
    }
    znak = fgetc(vstup);
    while (znak != EOF) {
        fputc(znak, vystup);
        znak = fgetc(vstup);
    }
    fclose(vystup);
    fclose(vstup);
    return 0;
}
```

Kopírování textového souboru po řádcích

```
#include <stdio.h>
#define SIZE 81
char buffer[SIZE]; // pamet na jeden radek
int lng = 0;
FILE *vstup, *vystup;

int main(int argc, char *argv[]) {
    if (argc != 3) { fprintf(stderr, "Použití: %s vstup výstup\n", argv[0]); return 1; }
    vstup = fopen(argv[1], "r");
    if (vstup == NULL) { fprintf(stderr, "Vstupní soubor %s nelze otevřít\n", argv[1]); return 2; }
    vystup = fopen(argv[2], "w");
    if (vystup == NULL) { fprintf(stderr, "Výstupní soubor %s nelze otevřít\n", argv[2]); return 3; }
}

while (fgets(buffer, SIZE, vstup) != NULL)
    fputs(buffer, vystup);

fclose(vystup);
fclose(vstup);
return 0;
}
```


Kopírování souborů binárně

```
#include <stdio.h>
#define SIZE 1024
unsigned char buffer[SIZE];
int lng = SIZE;
FILE *vstup, *vystup;

int main(int argc, char *argv[]) {
    if (argc != 3) {
        fprintf(stderr, "Použití: %s vstup výstup\n", argv[0]);
        return 1;
    }
    vstup = fopen(argv[1], "rb");
    if (vstup == NULL) {
        fprintf(stderr, "Vstupní soubor %s nelze otevřít\n", argv[1]);
        return 2;
    }
    vystup = fopen(argv[2], "wb");
    if (vystup == NULL) {
        fprintf(stderr, "Výstupní soubor %s nelze otevřít\n", argv[2]);
        return 3;
    }
    while (lng = fread(buffer, 1, SIZE, vstup)) fwrite(buffer, 1, lng, vystup);
    fclose(vystup);
    fclose(vstup);
    return 0;
}
```

Binární a textový režim

- Textový režim – vše jako text (i čísla) 1.23e07 => uložení 7 znaků.
- Binární režim – uložení jako např. 64-bitová hodnota `double`.
- Binární uložení a čtení je rychlejší – nejsou konverze, po blocích.
- Textové uložení – přenositelné lépe mezi systémy.
- Binární – pro přenos se musí napsat aplikace, která rozumí vnitřní reprezentaci.
- Příklad pro tabulku osob:

```
typedef struct Osoba{
    int ID;
    char jmeno[24];
    char prijmeni[35];
    char narozeni[10];
} OSOBA;

OSOBA tab[100];
int pocet = 0;
```

Uložení tabulky v textovém režimu

```
int ulozText(char *jmeno) {
    FILE *vystup;
    vystup = fopen(jmeno, "w"); // fopen_s(&vystup, jmeno, "w")
    int i;
    if (vystup == NULL) return 1;
    fprintf(vystup, "%d\n", pocet);
    for (i = 0; i < pocet; i++)
        fprintf(vystup, "%d \"%s\" \"%s\" \"%s\"\\n",
                tab[i].ID, tab[i].jmeno, tab[i].prijmeni, tab[i].narozeni);
    fclose(vystup);
    return 0;
};
```

```
typedef struct Osoba{
    int ID;
    char jmeno[24];
    char prijmeni[35];
    char narozeni[10];
} OSOBA;
```

```
OSOBA tab[100];
int pocet = 0;
```

Hlavní program v textovém režimu

```
void tisk(void) {
    int i;
    for (i = 0; i < pocet; i++)
        printf("%d \">%s\> \">%s\> \">%s\>\n",
            tab[i].ID, tab[i].jmeno, tab[i].prijmeni, tab[i].narozeni);
};

int main() {
    tab[pocet++] = { pocet+100, "Pavel", "Novak", "10.2.2004" };
    tab[pocet++] = { pocet+100, "Dana", "Novakova", "11.6.2005" };
    tab[pocet++] = { pocet+100, "Karel", "Polak", "8.12.2002" };
    tisk();
    ulozText("Osoby.txt");
    nactiText("Osoby.txt");
    tisk();
    system("pause");
    return 0;
};
```

Uložení tabulky v binárním režimu

```
int ulozBin(char *jmeno) {
    FILE *vystup;
    fopen_s(&vystup, jmeno, "wb");
    if (vystup == NULL) return 1;
    fwrite(tab, sizeof(OSOBA), pocet, vystup);
    fclose(vystup);
    return 0;
};

int nactiBin(char *jmeno) {
    FILE *vstup;
    fopen_s(&vstup, jmeno, "rb");
    if (vstup == NULL) return 1;
    pocet = fread(tab, sizeof(OSOBA), 100, vstup);
    fclose(vstup);
    return 0;
};
```

```
typedef struct Osoba{
    int ID;
    char jmeno[24];
    char prijmeni[35];
    char narozeni[10];
} OSOBA;
```

```
OSOBA tab[100];
int pocet = 0;
```

Hlavní program v binárním režimu

```
void tisk(void) {
    int i;
    for (i = 0; i < pocet; i++)
        printf("%d \"%s\" \"%s\" \"%s\"\\n",
            tab[i].ID, tab[i].jmeno, tab[i].prijmeni, tab[i].narozeni);
};

int main() {
    tab[pocet++] = { pocet+100, "Pavel", "Novak", "10.2.2004" };
    tab[pocet++] = { pocet+100, "Dana", "Novakova", "11.6.2005" };
    tab[pocet++] = { pocet+100, "Karel", "Polak", "8.12.2002" };
    tisk();
    ulozBin("Osoby.bin");
    nactiBin("Osoby.bin");
    tisk();
    system("pause");
    return 0;
};
```

Přímý přístup k souboru v C

- **Pozice a posun v souboru:**
 - Odkud a kam se bude zapisovat?
 - „Get pointer“ – místo, odkud se bude číst.
 - „Put pointer“ – místo, kam se bude zapisovat.
- **`ftell()` – získání pozice ukazatele do souboru.**
- **Nastavení pozice ukazatele pro vstup `fseek(offset, pozice_odkud)`:**
 - `pozice_odkud` určuje, zda se ukazatel nastaví vzhledem k:
 - začátek souboru (`SEEK_BEG`),
 - aktuální pozice (`SEEK_CUR`), nebo
 - konec souboru (`SEEK_END`).
 - `offset` určuje, o kolik se ukazatel posune vzhledem k pozici.
- **Počáteční pozice ovlivněna módem otevření.**

Přímý přístup – délka souboru

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    FILE * inp;
    long delka = 0;
    system("dir");
    inp = fopen("inputFile.txt", "rb");
    if (inp != NULL) {
        fseek(inp, 0, SEEK_END);
        delka = (long)ftell(inp);
        printf("Velikost je %ld\n", delka);
        fclose(inp);
    }
    else
        printf("Soubor se nepodarilo otevrit\n");
    system("pause");
    return 0;
}
```


Parametry programu

- Funkce **main** může být bez parametrů:

```
int main(void) { ... }
```

- Funkce **main** může mít parametry:

```
int main(int počet, char *slova[]) { ... }
```

- 1.parametr udává počet slov v příkazovém řádku (slova jsou oddělena mezerou)
- 2.parametr je seznam slov

Příklad: při vyvolání programu copy.exe

```
>copy muj.txt tvuj.txt
```

dostane funkce **main** parametry:

```
počet == 3
```

```
slova[0] == "copy"
```

```
slova[2] == "tvuj.txt"
```

```
slova[1] == "muj.txt"
```

```
slova[3] == NULL (0)
```

Parametry programu (pokr.)

Příklad: správné řešení programu copy.exe s parametry:

```
#include <stdio.h>

int main(int argc, char *argv[]) {
    if (argc != 3)
        printf("Chybne volani copy, „  
                spravne: copy vstup vystup\n");
    else
        printf("Spravne volani copy ve tvaru: copy %s %s\n",  
                argv[1], argv[2]);
    return 0;
}
```

Návratová hodnota funkce `main`

- Pokud skončí hlavní program úspěšně, měla by být funkce `main` zakončena příkazem: `return 0;`
- Pokud funkce `main` skončí bez tohoto příkazu, předpokládá se úspěšný konec (tj. jakoby se provedl příkaz `return 0;`). To platí jen pro funkci `main` a případně funkci, která vrací hodnotu typu `void`. Někteří autoři nedoporučují implicitní `return 0;` jako špatnou praxi.
- Pokud vrátí funkce `main` hodnotu 0 (ať implicitně nebo explicitně), je to interpretováno jako úspěšný konec programu. Vracená hodnota může být zpravidla prostředím dále využita.
- Jiné hodnoty než 0 mohou být interpretovány jako příznak chyby. Zaručené hodnoty jsou definovány v knihovně `stdlib.h` (`cstdlib`):

Hodnota	Popis
0	úspěšný konec
EXIT_SUCCESS	úspěšný konec
EXIT_FAILURE	neúspěšný konec, chyba

The End